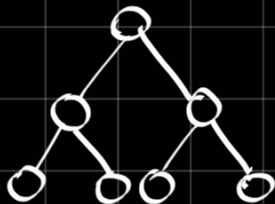
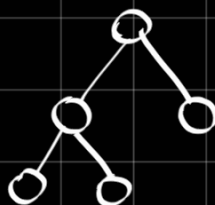


NOMENCLATURA E DEFINIZIONI UTILI:

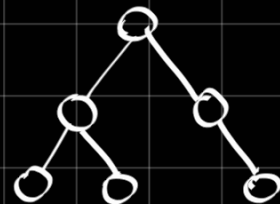
- **RADICE**: l'unico nodo dell'albero privo di arco entrante
- **FOGLIA**: un nodo senza archi uscenti
- **PADRE** di un nodo n : il nodo da cui l'arco entrante in n ha origine
- **FIGLIO** di un nodo n : il nodo in cui uno degli archi uscenti da n termina
- **ALBERO BINARIO**: Albero dove ogni nodo ha al più due figli
- **LIVELLO**: distanza, in numero di archi, di un nodo dalla radice (la radice sta al livello zero)
- **ALBERO BILANCIATO**: preso un qualsiasi nodo dell'albero, l'altezza dei suoi due sottoalberi differisce al massimo di 1 livello



PIENO



**PIENO E
COMPLETO**



NON **PIENO**

ALGORITMI DI VISITA DEGLI ALBERI:

INORDER(T)

```
1 if  $T \neq NIL$ 
2   INORDER( $T.left$ )
3   PRINT( $T.key$ )
4   INORDER( $T.right$ )
5 return
```

PREORDER(T)

```
1 if  $T \neq NIL$ 
2   PRINT( $T.key$ )
3   PREORDER( $T.left$ )
4   PREORDER( $T.right$ )
5 return
```

POSTORDER(T)

```
1 if  $T \neq NIL$ 
2   POSTORDER( $T.left$ )
3   POSTORDER( $T.right$ )
4   PRINT( $T.key$ )
5 return
```

- **BST**: Preso un qualsiasi nodo dell'albero, ogni nodo nel sottoalbero sinistro ha chiave più piccola, mentre ogni nodo nel sottoalbero destro ha chiave più grande.
Sia h l'altezza dell'albero (se l'albero è ben bilanciato allora $h = \log(n)$)...

OPERATIONE	COMPLESSITA'
Ricerca	$O(h)$
Min, Max	$O(h)$
Successore	$O(h)$
Insert, remove	$O(h)$

① Si scriva una procedura per costruire un BST ben bilanciato a partire da un array di n interi. Se ne valuti la complessità temporale asintotica

SOLUZIONE:

Per costruire l'albero, mi conviene inserire prima gli elementi più piccoli e poi quelli più grandi $\rightarrow$ il primo passo è ordinare l'array.

Successivamente, ragioniamo così: se voglio lo stesso numero di nodi in entrambi i sottoalberi della radice, possiamo dedurre che la chiave della radice è la mediana dell'array ordinato.

Quindi, possiamo costruire una procedura ricorsiva che divide ogni sottoarray a metà fino a quando non ho diviso tutti i nodi...

Scriviamo lo pseudocodice...

CREA BST (A):

MERGESORT (A)

return AUX (A, 0, n)

Funzione
ausiliaria che
crea il vero BST
ben bilanciato

AUX (A, i, j) :

if $i = j$:

return NIL

$m \leftarrow (i+j)/2$

$x \leftarrow \text{CREANODO}(A[m])^*$

$x.\text{left} \leftarrow \text{AUX}(A, i, m)$

$x.\text{right} \leftarrow \text{AUX}(A, m+1, j)$

return x

← Caso base: il sottoarray è grande 1. In questo caso restituiamo nulla

← Altrimenti, prendiamo l'elemento medio del sottoarray e lo indichiamo come radice del sottoalbero. I suoi figli sinistri e destri sono definiti richiamando la funzione AUX

* Supponiamo di avere la funzione CREANODO, che alloca lo spazio per il nodo dell'albero

Valutiamone la complessità :

$$T(n) = O(n \log(n) + \tilde{T}(n)) \quad \text{dove} \quad \tilde{T}(n) = 2\tilde{T}\left(\frac{n}{2}\right) + 1$$

Ordinamento array

chiamate ricorsive

la creazione del nodo ha costo costante

Siamo nelle condizioni per applicare il Master Theorem...

$$a=2 ; b=2 ; f(n) = 1 = O(n^{\log_b(a)})$$

$$\rightarrow \text{CASO } \textcircled{1} \rightarrow \tilde{T}(n) = O(n^{\log_b(a)}) = O(n)$$

$$\text{Quindi } T(n) = O(n \log(n) + n) = O(n \log(n))$$

- 2) Data un albero binario, dove non ci sono chiavi ripetute, e una coppia di interi a, b che rappresentano gli estremi di un intervallo, scrivere un algoritmo che stabilisca se si tratta di un BST oppure no, con tutte le chiavi in $[a, b]$
-

SOLUZIONE:

Per essere un BST allora per ogni nodo x , i suoi discendenti a sinistra devono avere chiave minore (o uguale) alla chiave di x , mentre i discendenti a destra devono avere chiave maggiore.

Ragioniamo per induzione: partendo dalle foglie, se un dato nodo x è figlio di un nodo y , allora ritorniamo false se:

- x è figlio dx di y e $x.key \leq y.key$
- x è figlio sx di y e $x.key > y.key$

Scriviamo, quindi, un algoritmo **DIVIDE ET IMPERA...**

check(T, a, b)

if ($T = NIL$) return true;

if ($T.left = NIL \ \&\& \ T.right = NIL$) return true

PASSO INDUTTIVO

return (check($T.left, a, T.key$) AND
check($T.right, T.key, b$) AND $a \leq T.key$
AND $T.key \leq b$)

Quanto vale la complessità?
Se l'albero ha n nodi allora...

$$T(n) = 2T\left(\frac{n}{2}\right) + k \Rightarrow T(n) = \Theta(n)$$

Due chiamate ricorsive
su metà dell'albero

Costo aggiuntivo
(costante)

Il caso pessimo è quando l'albero è veramente un
BST $\Rightarrow$ Controlla tutti i nodi!

3) Dato un albero binario, scrivere un algoritmo
per calcolare la somma delle chiavi delle foglie
dell'albero

SOLUZIONE:

Anche qui scriviamo un algoritmo **DIVIDE ET IMPERA**

CASO BASE: se il nodo in cui sto non ha figli
(o l'albero è degenero) allora ritorno
il valore della chiave

PASSO INDUTTIVO: chiedo la somma delle foglie dei
due sottoalberi e ritorno la loro somma

sommaFoglia(T):

```
if ( $T.left = NIL$  AND  $T.right = NIL$ ) return  $T.key$   
return sommaFoglia( $T.left$ ) + sommaFoglia( $T.right$ )
```

se n è il numero di nodi dell'albero allora, come prima...

$$T(n) = 2T\left(\frac{n}{2}\right) + k \rightarrow T(n) = \Theta(n)$$

VARIANTE: E se vogliamo la somma delle chiavi a livello " l "?

SOLUZIONE:

Dovremo tenere conto del livello a cui siamo
→ Inizializzo una variabile $a=0$. Se $a=l$, siamo nel caso base e ritorno la chiave del nodo corrente. Altrimenti, chiamo la stessa funzione sui sottoalberi

OPPURE: senza inizializzare a , a ogni ricorrenza faccio $l-1$.

sommaFoglia(T, l):

if ($l = 0$) return $T.key$

return sommaFoglia($T.left, l-1$) + sommaFoglia($T.right, l-1$)

Anche qui, $T(n) = \Theta(n)$

④ // // // algoritmo per
verificare se l'albero binario è completo
oppure no

SOLUZIONE:

Ricordiamo che, per avere un albero binario **COMPLETO**, ogni nodo deve avere un figlio destro e sinistro. Se tutte le foglie stanno sullo stesso livello, allora l'albero si dice **PERFETTO**

Casi particolari: (sarà il nostro caso base)

- L'albero vuoto è completo
- L'albero con solo il nodo radice è completo

isComplete(T):

if (T = NIL) return true

if (T.left = NIL AND T.right = NIL) return true

if (T.left ≠ NIL AND T.right ≠ NIL):

return isComplete(T.left) AND isComplete(T.right)

else return false

DOMANDA:

Se volessimo avere un albero perfetto?

In questo caso, nel codice avremmo dovuto contare il numero di nodi su ogni livello...

↙ (Esercizio da tema d'esame)

5

Si considerino alberi binari, in cui ciascun nodo contiene come informazione un numero intero. Si progetti un algoritmo che, ricevendo in ingresso un albero T e un intero tot , restituisce 1 se esistono almeno due cammini distinti dalla radice alle foglie in cui la somma dei numeri contenuti nei nodi è uguale a tot e 0 altrimenti.

Si calcoli l'ordine di grandezza (Θ) della complessità temporale dell'algoritmo proposto in funzione del numero n di nodi dell'albero.

SOLUZIONE:

Si può progettare un algoritmo ricorsivo in questo modo:

- Guardo il primo nodo k_1 : voglio verificare se esiste un percorso, andando a sinistra o a destra, che, sommato a k_1 mi dia tot
- Guardo il figlio sx, k_2 : voglio verificare se esiste un percorso, andando a sinistra o a destra, che, sommato a k_2 mi dia $tot - k_1$. faccio lo stesso con il figlio dx.
- Vado avanti fino al caso base (raggiungo una foglia): se il valore della foglia sottratto a $tot - k_1 - k_2 - \dots$ mi dà 0 allora ho trovato un percorso $\rightarrow$ lo aggiungo a una variabile che segna il numero di percorsi trovati. se il risultato è maggiore o uguale a 2 allora true

CERCA (A , tot):

AUX (A .root, tot)

if res $\geq$ 2

return 1

return 0

AUX (x, tot) :

if x = NIL

return 0

if x.left = NIL and x.right = NIL Se mi trovo in

if x.key = tot ← una foglia e la

return 1 sua chiave è uguale a tot

return 0

significa che ho trovato un

percorso

res1 ← AUX (x.left, tot - x.key)

res2 ← AUX (x.right, tot - x.key)

Altrimenti,

return res1 + res2

il percorso non è valido

Se non mi trovo in

una foglia, continuo a scendere, e se trovo

percorsi validi, li memorizzo nelle variabili

res1 e res2

Valutiamone la complessità, che sostanzialmente dipende da quella della funzione AUX.

AUX fa una visita in profondità di tutti i nodi dell'albero una sola volta. Le operazioni fatte su questi nodi sono a tempo costante. Dunque, la complessità dell'algoritmo è la stessa di quella della visita in ampiezza di un albero, cioè $\Theta(n)$

↙ (Esercizio da tema d'esame)

6

Si consideri un albero binario di ricerca T contenente $n > 0$ nodi. Si implementi in pseudocodice la procedura $\text{STAMPASTRATI}(T)$ che stampa il contenuto dell'albero per livelli, a partire dalla radice, stampando i nodi di ogni livello da sinistra a destra. Se ne analizzi la complessità temporale.

SOLUZIONE:

Non è possibile costruire un buon algoritmo ricorsivo che stampi i nodi dell'albero come vogliamo noi.

Una idea alternativa è utilizzare un'altra struttura dati, una **CODA**, per tenere traccia degli elementi sullo stesso livello e stamparli al momento giusto.

STAMPASTRATI (T):

```
enqueue(Q, T.root)
while ¬IsEmpty(Q)
    x ← dequeue(Q)
    stampa(x)
    enqueue(Q, x.left)
    enqueue(Q, x.right)
```

Ricorda che una coda è FIFO, quindi i primi elementi inseriti saranno stampati per primi

Anche qui, la complessità dell'algoritmo è $\Theta(n)$, perché visiteremo ogni nodo dell'albero una sola volta.